

The Import Graph Answers a Different Question

Structure, Issue Text, and What Belongs in a Context Window

NEIL LUNAVAT, Pimpri Chinchwad College of Engineering, India

An agent asked to change one file in an unfamiliar repository cannot read the repository. In our corpus the median repository is 286,385 tokens and 60.5% of tasks sit in one larger than a 200,000-token window; the files that actually matter are a median 2.98% of it. Something must choose. Two signals are available for the choice: the repository's import structure, and the words of the issue report. Tools overwhelmingly use the first.

We compare both, and their combination, on 2,412 change tasks from 1,162 real pull requests in 112 Python repositories, scoring each ordering by how much of an evidence-based answer key it covers per line read. The comparison only has an answer once one says what "needed" means, and that turns out to decide everything. Graded against the files *co-edited* in the same pull request, the four best orderings fuse structure with issue text, the best pure signal is issue-text-to-path matching (AUC 0.436), and the best purely structural ordering is tenth (0.389); a query-independent PageRank prior is seventeenth, barely above random (0.272 vs. 0.247). Graded against the files that *define symbols the change uses*, the ranking inverts: cost-scaled personalised PageRank is first (0.489), issue-text matching falls to thirteenth (0.383), and the PageRank prior climbs to tenth, far above random (0.410 vs. 0.251). Across the sixteen contending methods the two evidence sources induce rankings that are statistically unrelated (Spearman $\rho = 0.26$, $p = 0.34$). Every reversal is significant under Holm correction with the pull request as the unit. We verify that the four graph-based localization systems we could check all grade against the first of these keys — the one on which structure is weakest.

Two further results bound the problem. The two-hop neighbourhood of the seed, the usual justification for graph retrieval, holds 87.5% of the needed files but is also 56% of the repository and a median of 21,646 lines: it supplies recall, not precision, at a lift of only 1.57 $\times$ over taking the same share of files at random. And the headroom is large and measurable without running any model: an oracle ordering puts the complete answer inside an 8,000-line budget for 97.5% of pull requests, whereas the best method we test reaches 58.7%. Since no agent can use a file it was never given, that gap bounds every downstream system at once. Reciprocal-rank fusion of one structural and one lexical ordering is the only family in the top six under both evidence sources, gaining 0.069 AUC over pure structure on co-edited files ($p < 0.001$) while losing nothing on symbol files (-0.0004 , $p = 1.0$). We argue that retrieval for code change must be reported per evidence source, and that a cheap lexical signal belongs in every graph-based context tool. We make no claim about downstream repair rates; Section 6 pre-registers the experiment that would test one.

Additional Key Words and Phrases: code search, context retrieval, dependency graphs, software maintenance, empirical software engineering

ACM Reference Format:

Neil Lunavat. 2026. The Import Graph Answers a Different Question: Structure, Issue Text, and What Belongs in a Context Window. *ACM Trans. Softw. Eng. Methodol.* 1, 1 (September 2026), 16 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

An agent is handed a bug report and a file to change. Before it can write anything it must decide what else to read, and it cannot simply read everything. In the corpus we study the median repository is 286,385 tokens; 74.8% of tasks sit in a repository larger than a 128,000-token window and 60.5% in one larger than 200,000. Meanwhile the files that carry the answer are a median 2.98% of the repository. The reduction available is roughly 34 $\times$, and every line spent on an irrelevant file is

Author's address: Neil Lunavat, Pimpri Chinchwad College of Engineering, India, neillunavat3192@gmail.com.

2026. 1049-331X/2026/9-ART \$15.00
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

a line not spent on a relevant one. Something has to choose, and the choice is the subject of this paper.

A human faces a softer version of the same problem — attention rather than tokens — and we return to that reading in Section 5. The budget is what makes the question sharp, so we pose it in those terms throughout.

The dominant answer in tool building is the dependency graph. Aider builds a repository map from the call and definition graph; RepoGraph, LocAgent and CoSIL construct code graphs and walk them outward from a starting point. The premise is intuitive and rarely stated: what you need to read is near, in the graph, to what you are changing.

There is a second signal, and it is the one a human reaches for first. Asked to fix the login page, a developer thinks of the authentication files — not because `login.py` imports them, which it may well not, but because the words match. That signal sits in the issue report, and graph-based tools largely do not use it for ordering.

This paper compares the two. The comparison sounds simple and is not, because it has no answer until one says what it means for a file to have been “needed”. We use two evidence sources that can be recovered from the repository without human labelling: the files *co-edited* in the same pull request, and the files that *define symbols* the changed code references. Both are defensible. Neither is obviously the right one. They give opposite answers to the question this paper asks.

Research questions

- RQ1** How much of a repository is the seed’s two-hop neighbourhood, and how much recall does it buy over a same-sized random sample?
- RQ2** Do structural and lexical reading orders rank differently under the two evidence sources, and by how much?
- RQ3** How much does conditioning on the seed file add over a query-independent repository prior, and does the answer depend on the evidence source?
- RQ4** Do the two signals retrieve the same files, or different ones?
- RQ5** Does fusing them produce an ordering that is good under both evidence sources?
- RQ6** What fraction of tasks could be solved at all under a given budget — that is, how much of the gap between current methods and a perfect ordering is headroom rather than a property of the tasks?

Contributions

- (1) A head-to-head comparison of import-graph proximity and issue-text retrieval as *reading orders* under a line budget, on 2,412 tasks from 1,162 real pull requests in 112 Python repositories, with both signals scored against the same two evidence sources.
- (2) Evidence that the conclusion of such a comparison is decided by the evidence source rather than by the methods: sixteen methods, two sources, rankings unrelated at $\rho = 0.26$.
- (3) A quantification of what graph proximity actually contributes: recall over half the repository, at a lift of 1.57×, with 4.2% precision at two hops.
- (4) A robustness criterion — worst-case rank across evidence sources — and the finding that under it every fusion of a structural and a lexical ordering beats every pure signal.
- (5) A cost accounting in tokens rather than files: a corpus-wide calibration of Python source (8.50 tokens per line in aggregate, median 7.88), and a method-independent ceiling — the share of pull requests whose complete answer fits a given budget at all. An oracle ordering reaches 97.5% of pull requests within 8,000 lines where the best method we test reaches 58.7%, so the shortfall is headroom and not task difficulty.

- (6) A replication package with the graphs, tasks, issue-token bags, token counts, orderings and analysis scripts.

2 BACKGROUND AND RELATED WORK

Dependency structure as a guide to change. Using program structure to decide what a change touches is old. Impact analysis [3] propagates from a changed element along dependencies; Robillard [18] ranks program elements by the topology of their dependencies; feature-location surveys [6] catalogue structural, textual and dynamic techniques and note that they are usually evaluated in isolation. Ko et al. [10] observed developers navigating largely by identifier names and search rather than by following dependencies.

Change history. Zimmermann et al. [26] and Ying et al. [23] mine co-change from version histories to suggest files that change together. Our co-edited evidence source is the same notion of relatedness, used as ground truth rather than as a method.

Graph-based context for agents. SWE-bench [9] and its successors [2, 24] made repository-scale change tasks a standard benchmark, and a family of systems now assembles context by walking a code graph: Aider’s repository map [1], RepoGraph [14], LocAgent [5] and CoSIL [8]. Agentless [20] is a notable counterexample that leans on file and directory names.

What these systems share is the ground truth they are judged against. Table 1 records it for every system whose evaluation section we read in the published PDF; each entry is quoted or paraphrased from that section, not from an abstract or a secondary description. Where localization accuracy is reported at all, the target is the set of files or functions touched by the gold patch — the SWE-bench convention, and exactly our co-edited source. That is the evidence source on which, in our results, structural proximity is the weaker of the two signals.

Concurrent work is questioning the key. The patch convention is under active revision, by at least four efforts that appeared while this study was running, and we want to be exact about how close they come. SWE-Explore [25] derives line-level ground truth “from independent agent trajectories that successfully solved the same issue, distilling the specific code regions their solution paths actually consulted”, and scores a ranked list “under a fixed line budget” — both the key and the cost model are close relatives of ours. ContextBench [11] annotates gold context by hand, 522,115 lines over 1,136 tasks, and measures recall, precision and efficiency rather than end-to-end resolution. AgentRetrievalBench [15] defines a file as relevant “when reading it would materially help the agent take the next correct step”, evaluated under a top- k or token budget; one of its four task types, ripple files beyond an anchored edit, is our co-edited source by another name. And GraphLocator [12] keeps the patch key but dual-annotates it, on the stated grounds that patch-derived ground truth misses locations.

We take this as corroboration of the premise rather than as a threat to it: several groups independently concluded in the same year that the patch key is inadequate. What none of them reports is the consequence we measure. Each proposes a better key and evaluates methods against it; we hold two keys already recoverable from existing benchmark data, and show that the choice between them *reverses the ranking of the methods themselves* — sixteen methods, rankings uncorrelated at $\rho = 0.26$, every individual reversal significant. The contribution is not the observation that keys differ. It is the measurement of what the field’s default key costs the signal the field has invested in.

Retrieval. BM25 [17] remains the reference lexical ranking function; we use it unchanged, varying only the query.

Table 1. What repository-context systems are graded against, read from each paper’s own evaluation section (2026-09-23). The upper block is the patch-based convention; the lower block is the concurrent work that departs from it.

System	Venue	Ground truth used for evaluation
<i>Files touched by the gold patch</i>		
LocAgent [5]	ACL 2025	“The affected files or functions ...as identified in the patches, are considered the target locations”
CoSIL [8]	ASE 2025	File- and function-level accuracy on SWE-bench Lite and Verified, i.e. the patched locations
Agentless [20]	FSE 2025	Correct if the edit covers a superset of the ground-truth patch locations, at line, function and file level
LingmaAgent [13]	FSE 2025	Fault locations extracted from the developer patch, at file and function level
SWE-Fixer [21]	ACL 2025	Precision and recall against the defective files of the resolving pull request; BM25 top-30 then a learned reranker
KGCompass [22]	2025	Files and code elements modified by the ground-truth patch, scored by Jaccard overlap
SWE-Adept [7]	2026	Correct only if <i>all</i> locations modified by the ground-truth patch appear in the top- <i>k</i>
SWERank [16]	2025	Every function modified in the pull request is a positive
CodeScout [19]	2026	Reuses LocAgent’s patch-processing scripts: modified files, modules and functions
RepoGraph [14]	ICLR 2025	“The percentage of problems where the edit locations match the ground truth patch”, at file, function and line level
<i>Something other than the gold patch alone</i>		
GraphLocator [12]	2025	Locations extracted from the fix patch, then dual-annotated and cross-reviewed because patch-derived ground truth “contains missing locations”
SWE-Explore [25]	2026	File and line intervals that successful agents actually <i>read</i> , intersected across several verified trajectories
ContextBench [11]	2026	Human-annotated context traced outward from the patch regions, pruned by a second annotator, validated by an LLM patching from it alone
AgentRetrievalBench [15]	2026	A file is relevant if reading it helps the agent’s next step; one of four task types is ripple files beyond an anchor
CodeR [4]	2024	End-to-end on SWE-bench, but its separate localization study uses golden locations marked by the authors

3 STUDY DESIGN

The corpus, the graph builder, the task definition and the coverage-cost metric were fixed and tagged in version control before any method was run; the present study adds method families and analyses on top of that frozen base. Changes made after that tag, and claims we have retracted, are recorded in the replication package rather than silently absorbed.

3.1 Repositories and tasks

We draw pull requests from SWE-bench, SWE-bench-Live, SWE-Gym, SWE-bench Pro, Loc-Bench and SWE-rebench, keeping Python repositories with a permissive licence. A pull request that edits *k* existing non-test source files that are nodes of the import graph yields *k* tasks: each edited file in turn becomes the *seed*, and the task is to order the remaining files of the repository. This gives 3,123 tasks from 1,873 pull requests in 113 repositories. Of these, 2,412 tasks — 1,162 pull requests in 112 repositories — have at least one key file inside the graph once the seed is removed, and they are the ones scored here; the remaining 711 tasks have an empty key under both sources and are

reported but not scored. Within the scored set, 1,958 tasks (708 pull requests) have a co-edited key and 2,341 (1,156 pull requests) a symbol key.

The median repository contributes 131 graph nodes and the median key is 3 files totalling 1,515 lines, so the task is to find a handful of files among roughly a hundred.

3.2 Import graphs

Graphs are built per base commit with a custom ast walker over the repository at that commit. Nodes are source files; an edge $a \rightarrow b$ means a imports a module resolving to b . Package `__init__` files are not inserted as intermediaries, from `X import Y` resolves to the module, edges are unweighted, imports inside `TYPE_CHECKING` blocks and inside function bodies are kept, and test files are excluded. Per-file line counts come from the same pass and are the cost unit throughout.

3.3 Two evidence sources

co_edited The other non-test source files changed by the same pull request. This is what a developer in fact touched, and it is independent of the import graph.

symbol Files that define names the changed code references, resolved with a static analyser. This is what the change depends on.

The seed file is removed from both. We deliberately do not merge them: the whole question is whether the choice matters. It does, and the union is not a neutral compromise — the symbol key contributes 7,675 file references against 4,458 for co-editing, so a union largely inherits the structural source’s verdict.

A third source — files actually executed by the tests that the fix repairs — is defined in our protocol and was collected for 113 pull requests before we stopped. We exclude it rather than report it: a source covering under a tenth of the corpus cannot be compared against two that cover all of it, and reporting it alongside them would invite exactly the conflation this paper argues against. The partial collection is in the replication package so that it is visible rather than quietly dropped, and completing it is the obvious extension.

3.4 Reading orders

Every method receives the graph at the base commit, the seed file, per-file identifier bags and the issue text, and returns an ordering of all other nodes. Table 2 lists them by family. Identifier bags are built by splitting every identifier in the file on underscores and camel-case boundaries and lower-casing; the issue text is tokenised identically, so text is matched against code over one vocabulary. BM25 is used with the standard $k_1 = 1.5$, $b = 0.75$ and differs between methods only in the query. Fusion is reciprocal-rank fusion with $k = 60$.

Two points of fairness are worth stating. The structural methods are *given* a correct answer for free — the seed is one of the files the pull request changed. The issue-text methods are given no seed at all. And the metric charges by lines read, so we include cost-scaled variants (suffix `_p1`, score divided by file length) to separate “ranks the right files highly” from “happens to prefer short files”.

3.5 Metric

An ordering is read from the top until a line budget is exhausted; coverage is the fraction of the key that has been read. We evaluate at budgets $\{250, 500, 1000, 2000, 4000, 8000, 16000, 32000\}$ and report the AUC, the mean coverage over those eight doublings, so that a method must be good at both tight and generous budgets. We also report the more directly interpretable quantity: the number of lines that must be read before half the key has been covered.

Table 2. Reading orders, by family. Structural methods use the seed and the graph; global methods use neither the seed nor the issue; lexical methods use no graph.

Family	Method	Ordering
structure	bfs_und	breadth-first from the seed, undirected
	hops_lines	by (hop distance from seed, file length)
	ppr_und	personalised PageRank, restart at the seed
	ppr_und_pl	the same, divided by file length
global	pagerank	PageRank of the whole graph; ignores seed and issue
	indegree	number of files importing this one
lexical	bm25_seed	BM25, query = identifiers of the seed file
	path_seed	BM25 over paths, query = the seed's path tokens
	bm25_issue	BM25, query = identifiers of the issue text
	bm25_issue_pl	the same, divided by file length
	path_issue	BM25 over file paths, query = issue tokens
fusion	rrf_hops_path	RRF of hops_lines and path_issue
	rrf_hops_issue_path	RRF of hops_lines, bm25_issue, path_issue
	rrf_pprpl_issue_path	RRF of ppr_und_pl, bm25_issue, path_issue
	rrf_ppr_issue	RRF of ppr_und and bm25_issue
	rrf_ppr_issue_path	RRF of ppr_und, bm25_issue, path_issue
reference	same_dir	files in the seed's directory first
	random	uniformly shuffled: the floor
	oracle	key files only, cheapest first: the ceiling

Orderings with ties are re-drawn three times per task with distinct seeds and the coverage curves averaged.

The budget is denominated in lines because the pre-registered design fixed it that way at freeze-1, and we do not change a frozen metric after seeing results. Lines are nevertheless the wrong unit for a context window, so we measure the conversion rather than assume it: across the 121,075 distinct file versions in the corpus, tokenised with the c1100k_base byte-pair vocabulary, Python source runs to 8.50 tokens per line in aggregate and a median of 7.88 per file, with a 5th–95th percentile range of 5.70–10.90. Absolute counts shift by a small constant factor between tokenizer families; the ratios we report do not depend on the choice. A budget of 8,000 lines is therefore roughly 68,000 tokens, and the eight budgets span roughly 2,100 to 272,000 tokens — from a fraction of a small context window to more than a large one.

3.6 Statistics

The unit of analysis is the pull request: task AUCs are averaged within a pull request first, so a change touching many files does not count many times. Paired comparisons use the Wilcoxon signed-rank test with the Vargha–Delaney A_{12} effect size, Holm-corrected within each family of comparisons. Confidence intervals are 2,000-resample bootstraps resampling *repositories*, not tasks, so that a few large repositories cannot manufacture significance.

Table 3. Where the key files sit relative to the seed, and how big the two-hop ball is (medians over 2,412 tasks).

Key files at hop distance	1	2	3+	unreachable
co_edited	39.0%	49.1%	9.1%	2.8%
symbol	45.3%	41.4%	9.1%	4.2%
<i>The two-hop ball: 68 files, 21,646 lines, 56% of the repository</i>				
recall 87.5%, precision 4.2%, recall lift over a same-sized random sample 1.57×				
<i>The key: 3 files, 1,515 lines</i>				

4 RESULTS

4.1 RQ1: the two-hop neighbourhood is half the repository

“The files you need are within two hops” is true here: 86.7% of symbol-key files and 88.1% of co-edited files lie within two undirected hops of the seed (Table 3). Taken alone this reads as strong evidence for graph retrieval.

The other half of the statistic undoes it. In the median task the two-hop ball holds 68 of the repository’s 131 files and 21,646 lines — 56% of the files on average, and more than five times a 4,000-line budget. Its precision is 4.2%. Against a random sample of the same size it lifts recall by a factor of 1.57.

The two-hop neighbourhood is therefore not a filter. It is a statement that the files you need are somewhere in about half the repository — which is exactly the regime in which the *ordering* decides everything and the graph has already said all it has to say.

4.2 RQ2: the evidence source decides the winner

Table 4 is the central result. Under the co-edited key the top five orderings are four fusions and one lexical method; the best purely structural ordering is tenth and the query-independent priors are seventeenth and eighteenth, below *same_dir*. Under the symbol key the table inverts: cost-scaled personalised PageRank is first, three of the top four are structural, and the best lexical ordering is thirteenth.

The individual reversals are large and all significant after Holm correction. Issue-text path matching beats personalised PageRank by 0.078 AUC on co-edited files ($A_{12} = 0.597$, $p < 0.001$) and loses to it by 0.062 on symbol files ($A_{12} = 0.417$, $p < 0.001$). BM25 on the issue text beats it by 0.050 and then loses by 0.105 ($A_{12} = 0.364$).

Across the sixteen contending methods the two sources induce rankings that are not significantly related (Spearman $\rho = 0.26$, $p = 0.34$; including the three references, which sit at the same end of both rankings by construction, raises this only to $\rho = 0.45$, $p = 0.06$).

Figure 1 shows the same reversal as coverage curves. Table 5 states it in the unit a context budget is actually spent in: to reach half the co-edited files a reader following personalised PageRank consumes a median of 6,514 lines, against 4,747 following issue-text path matching and 3,254 following the best fusion. On symbol files the same three numbers are 4,395, 6,524 and 3,849.

4.3 RQ3: what the seed is worth depends on the source too

A query-independent PageRank prior — a ranking that never looks at the seed or the issue, and simply says which files matter in this repository — reaches 0.410 on symbol files against 0.446 for personalised PageRank, which does look at the seed. That is 92% of the personalised score, and

Table 4. Mean AUC by evidence source, pull request as the unit, with bootstrap 95% confidence intervals over repositories. The same nineteen orderings, ranked twice.

co_edited (708 PRs)				symbol (1,156 PRs)			
#	Method	Fam.	AUC [CI]	#	Method	Fam.	AUC [CI]
1	oracle	ref	0.773 [.750,.796]	1	oracle	ref	0.793 [.772,.813]
2	rrf_pprpl_issue_path	fus	0.488 [.457,.520]	2	ppr_und_pl	str	0.489 [.460,.517]
3	rrf_hops_issue_path	fus	0.482 [.450,.513]	3	rrf_hops_path	fus	0.477 [.450,.504]
4	rrf_hops_path	fus	0.458 [.426,.489]	4	hops_lines	str	0.477 [.447,.505]
5	rrf_ppr_issue_path	fus	0.446 [.415,.477]	5	rrf_pprpl_issue_path	fus	0.475 [.447,.503]
6	path_issue	lex	0.436 [.406,.466]	6	rrf_hops_issue_path	fus	0.458 [.430,.486]
7	bm25_issue	lex	0.407 [.379,.435]	7	ppr_und	str	0.446 [.417,.475]
8	bm25_seed	lex	0.393 [.362,.425]	8	rrf_ppr_issue_path	fus	0.421 [.392,.450]
9	rrf_ppr_issue	fus	0.393 [.363,.425]	9	bfs_und	str	0.420 [.391,.447]
10	hops_lines	str	0.389 [.354,.423]	10	pagerank	glo	0.410 [.381,.439]
11	ppr_und_pl	str	0.371 [.338,.403]	11	indegree	glo	0.398 [.371,.425]
12	path_seed	lex	0.368 [.335,.401]	12	rrf_ppr_issue	fus	0.391 [.361,.421]
13	bfs_und	str	0.362 [.330,.395]	13	path_issue	lex	0.383 [.352,.414]
14	ppr_und	str	0.357 [.325,.391]	14	path_seed	lex	0.365 [.335,.393]
15	bm25_issue_pl	lex	0.354 [.318,.389]	15	bm25_issue_pl	lex	0.355 [.324,.386]
16	same_dir	ref	0.327 [.296,.359]	16	bm25_issue	lex	0.340 [.311,.368]
17	pagerank	glo	0.272 [.240,.302]	17	bm25_seed	lex	0.336 [.305,.365]
18	indegree	glo	0.268 [.238,.299]	18	same_dir	ref	0.319 [.290,.348]
19	random	ref	0.247 [.218,.276]	19	random	ref	0.251 [.223,.278]

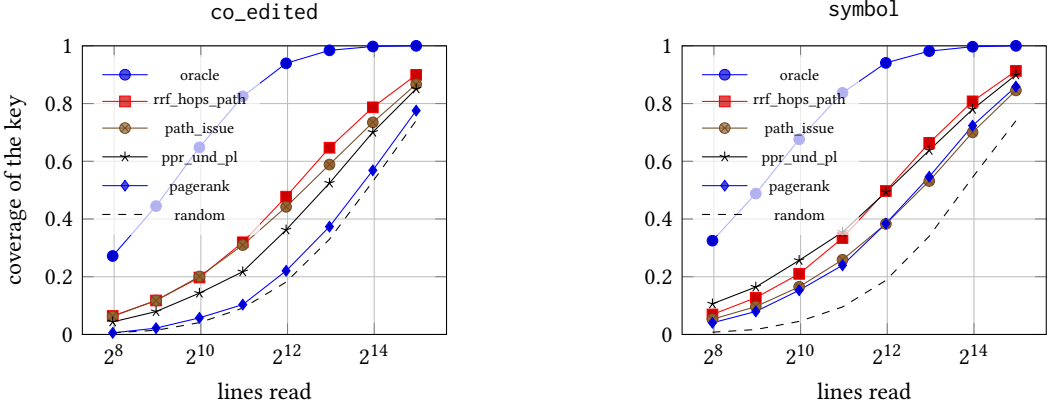


Fig. 1. Mean coverage against lines read. The same five orderings change places between the two evidence sources: issue-text path matching tracks the best fusion on co-edited files and falls below the query-independent prior on symbol files, while cost-scaled personalised PageRank does the reverse.

far above random (0.251; $A_{12} = 0.711$, $p < 0.001$). On that evidence source, most of what the personalised walk delivers is a repository prior.

Under the co-edited key the same prior scores 0.272 against a random floor of 0.247. The difference is significant but negligible ($A_{12} = 0.530$, $p = 0.004$), and personalised PageRank beats it by 0.086 ($A_{12} = 0.618$). Knowing which file is being changed therefore matters a great deal for predicting what else was changed, and rather little for predicting what the code calls, where knowing the repository's hubs is nearly as good.

Table 5. Median lines that must be read before half the key is covered. Lower is better.

Method	co_edited	symbol
oracle	524	440
rrf_pprpl_issue_path	3,254	3,849
rrf_hops_path	3,956	3,699
ppr_und_pl	6,155	3,483
hops_lines	5,620	4,080
ppr_und	6,514	4,395
path_issue	4,747	6,524
bm25_issue	5,012	7,451
pagerank	11,294	5,635
random	14,627	14,223

Table 6. How each key file is reached within the top k of the structural family (ppr_und, hops_lines) and the lexical family (bm25_issue, path_issue).

Source	k	both	structure only	words only	neither
co_edited	10	32.2%	11.4%	30.0%	26.4%
co_edited	20	50.7%	9.6%	24.4%	15.3%
co_edited	50	76.7%	5.9%	13.4%	4.0%
symbol	10	28.5%	23.3%	20.4%	27.8%
symbol	20	47.4%	19.9%	16.5%	16.2%
symbol	50	72.9%	11.5%	9.8%	5.9%

This is worth stating plainly, because the opposite claim is easy to make from a single source: whether “the graph knows what you are working on” appears true or false is itself a function of the answer key.

4.4 RQ4: the two signals name different files

The reversal is not two methods disagreeing about the order of one list. At $k = 10$ the top-10 sets of personalised PageRank and issue-text BM25 have a mean Jaccard overlap of 0.187, and of personalised PageRank and issue-text path matching only 0.102; at $k = 50$ these rise to 0.508 and 0.412. For comparison, personalised PageRank and the query-independent prior overlap at 0.364 at $k = 10$: a seed-conditioned walk resembles the repository prior more than it resembles a lexical ranking.

Table 6 follows individual key files. At $k = 20$, 24.4% of co-edited files are found only by the lexical family and 9.6% only by the structural family; on symbol files the asymmetry flips (19.9% structure-only against 16.5% words-only). Under either source about a sixth of key files are found by neither family at $k = 20$, which is where the remaining headroom to the oracle lives.

4.5 RQ5: fusion is the only ordering that survives both sources

If the evidence source is not known in advance — and for a tool shipped to users it is not — the quantity to optimise is the worst rank a method attains across sources. Table 7 ranks by it. Every one of the five fusions beats every pure signal. rrf_hops_path, which is nothing more than reciprocal-rank fusion of hop-distance ordering with issue-tokens-against-file-paths, is fourth under one source and third under the other.

Table 7. Worst rank attained across the two evidence sources. Lower is better. Every fusion beats every pure signal.

Method	co_edited	symbol	worst
rrf_hops_path	4	3	4
rrf_pprpl_issue_path	2	5	5
rrf_hops_issue_path	3	6	6
rrf_ppr_issue_path	5	8	8
rrf_ppr_issue	9	12	12
hops_lines	10	4	10
ppr_und_pl	11	2	11
bfs_und	13	9	13
path_issue	6	13	13
ppr_und	14	7	14
path_seed	12	14	14
bm25_issue_pl	15	15	15
bm25_issue	7	16	16
pagerank	17	10	17
bm25_seed	8	17	17
indegree	18	11	18

Table 8. Share of pull requests whose *entire* key fits within a token budget. This is a ceiling: no ordering can exceed it, and no agent can act on files outside it.

Token budget	co_edited (%)	symbol (%)
4,000	34.1	31.5
8,000	54.2	50.8
16,000	75.1	72.5
32,000	90.3	87.3
64,000	96.9	95.5
128,000	99.5	99.4
200,000	99.9	99.9

The insurance is close to free. Against pure structure (hops_lines) it gains 0.069 AUC on co-edited files ($A_{12} = 0.586$, $p < 0.001$) and loses 0.0004 on symbol files ($p = 1.0$); against ppr_und_pl it gains 0.087 ($p < 0.001$) and loses 0.012 ($p = 0.47$). The lexical signal adds a large amount where structure is weak and costs nothing where structure is strong.

4.6 RQ6: the shortfall is headroom, not task difficulty

A coverage number says how much of the key a method found. It does not say whether finding all of it was possible. If the needed files together exceed the budget, no ordering of them fits and no downstream system can succeed, however capable: a model cannot use a file it was never given. That bound is a property of the task and the budget alone, so it can be measured once and applied to every method at once — and, usefully here, without running a model.

Table 8 reports it. The complete answer fits a 32,000-token budget for 90.3% of pull requests under the co-edited key and 87.3% under the symbol key, and a 64,000-token budget for over 95% of both. The task is not, in general, impossible at realistic budgets.

Table 9. Share of pull requests taken to *complete* coverage within 8,000 lines. The oracle row is the ceiling of Table 8 expressed in the frozen unit. The reversal of Table 4 reappears under this stricter criterion.

Ordering	co_edited (%)	symbol (%)
oracle	97.5	95.9
rxf_hops_issue_path	58.7	49.0
rxf_hops_path	54.6	50.4
bm25_issue	51.8	36.9
path_issue	48.4	36.5
hops_lines	48.1	51.0
ppr_und_pl	45.1	50.0
pagerank	31.8	43.4
same_dir	27.8	23.8
random	14.5	14.0

Against that ceiling, current orderings fall a long way short. Table 9 counts the pull requests each method takes to *complete* coverage at 8,000 lines, a budget at which an oracle succeeds on 97.5% of them. The best method reaches 58.7% on the co-edited key and 50.4% on the symbol key. Roughly forty points of the gap is therefore attributable to the ordering rather than to the budget or the task.

Two things are worth noting. The reversal survives the stricter criterion: the two issue-text methods lose about fifteen points moving from the co-edited key to the symbol key, while the two structural methods gain three to five, and pagerank gains eleven. And the fusion orderings again sit at or near the top of both columns, which is the same conclusion RQ5 reached by AUC, reached here by a measure that ignores partial credit entirely.

4.7 Sensitivity: are the lexical methods just copying paths out of the issue?

Issue reports often contain stack traces, and a stack trace names files. If the lexical advantage came from reports quoting the paths of the files that had to change, it would be an artefact of the benchmark rather than a property of language.

It does not. The full path of a key file appears verbatim in the issue text for 4.2% of key files, and 12.6% of tasks contain at least one; the file stem appears for 20.4%. Repeating the analysis on the 87.4% of tasks whose issue quotes no key path leaves the ordering essentially unchanged: path_issue falls from 0.436 to 0.409 on co-edited files but keeps its position ahead of every structural method, and the fusions keep the top four places. The comparable figures for the seed itself are 10.3% full path and 35.1% stem.

The path_seed control separates “issue text” from “path matching” more directly: the same path-BM25 ranking, queried by the seed’s own path instead of the issue, scores 0.368 on co-edited files against 0.436 for the issue query (+0.067, $p < 0.001$), and the two are indistinguishable on symbol files (+0.018, $p = 0.13$). The gain is attributable to the issue text, and only where the target is what developers changed.

4.8 Sensitivity: does the reversal survive in large repositories?

The corpus skews small — the median repository contributes 131 graph nodes — so the reversal has to be shown not to be an artefact of repositories one can almost read whole. Table 10 repeats the ranking within quartiles of repository size (quartile boundaries 73, 131 and 246 nodes; the largest repository has 2,487).

Table 10. Mean AUC (rank among the sixteen contenders) by quartile of repository size in graph nodes. Q1 ≤ 73 , Q2 ≤ 131 , Q3 ≤ 246 , Q4 up to 2,487.

Method	co_edited				symbol			
	Q1	Q2	Q3	Q4	Q1	Q2	Q3	Q4
rrf_hops_path	.554 (3)	.466 (3)	.396 (3)	.332 (3)	.563 (4)	.476 (2)	.408 (2)	.363 (3)
ppr_und_pl	.497 (11)	.399 (8)	.316 (10)	.241 (11)	.568 (3)	.467 (3)	.423 (1)	.387 (1)
hops_lines	.517 (6)	.412 (6)	.329 (8)	.244 (10)	.569 (2)	.454 (5)	.402 (4)	.343 (6)
ppr_und	.485 (13)	.373 (13)	.293 (13)	.231 (12)	.521 (7)	.418 (8)	.375 (6)	.344 (5)
path_issue	.538 (5)	.439 (5)	.374 (5)	.304 (5)	.516 (10)	.419 (7)	.324 (9)	.256 (10)
bm25_issue	.516 (7)	.402 (7)	.334 (6)	.287 (6)	.476 (13)	.367 (13)	.273 (13)	.233 (12)
pagerank	.442 (15)	.299 (15)	.200 (15)	.113 (15)	.520 (8)	.371 (12)	.334 (8)	.286 (9)

The reversal holds in every quartile and widens with size. In the largest quartile, issue-text path matching is fifth on co-edited files and tenth on symbol files, while cost-scaled personalised PageRank is eleventh and first respectively. The query-independent prior degrades fastest of all: on co-edited files in the largest quartile it scores 0.113 against a random floor of 0.089, which is to say that in the repositories where retrieval matters most, knowing only which files are central is worth almost nothing. rrf_hops_path is third or better in all eight cells.

5 DISCUSSION

The graph and the issue answer different questions. Read together, the results have a simple interpretation. An import edge records that one file calls another. That is close to the definition of the symbol key, and structural methods duly win there. The co-edited key records what a person decided to change in one sitting — a decision made about a feature, a bug, a concept — and words track that better than call structure does. Neither result is a defect in the other method. They are answers to different questions that the literature has been treating as one question.

The field's default key is the one structure is worst at. Every graph-based localization system we could check (Table 1) is graded against the files the gold patch touched — our co_edited source. On that source, in our measurements, the best purely structural ordering is tenth of sixteen and is beaten by matching issue words against file paths, a method with no graph, no model and no seed. This does not mean those systems are wrong; they are LLM agents that also read the issue, and the graph is one component among several. It does mean that the component they advertise is, on their own choice of ground truth, not the component doing the work, and that nobody has reported the comparison.

The practical consequence is not “use words instead”. It is that a tool whose ordering rests on one signal is being tuned against whichever evidence source its evaluation happened to adopt. The fusion results show the cost of insurance is near zero: adding a BM25 ranking of issue tokens against file paths to an existing graph walk needs no model, no index beyond the file list and no extra hop through the graph, and it is the difference between being tenth and being fourth when the target turns out to be co-editing.

What “within two hops” is worth. The two-hop neighbourhood earns its reputation on recall and loses it on precision: it contains seven of every eight needed files and also half the repository. Reporting the recall without the size is, on this corpus, reporting half a statistic.

What this means for an agent's context, and what it does not. The budget framing is not decoration: at a median 286,385 tokens, most of these repositories do not fit in the windows current systems run

with, so selection is forced rather than optional. Two numbers bound what selection can achieve. The answer is a median 2.98% of the repository, so a well-chosen context could be some 34× smaller than the whole; and an oracle ordering fits the complete answer inside 8,000 lines for 97.5% of pull requests where our best method manages 58.7%, so most of that reduction is currently unrealised.

We are deliberately not claiming that better selection raises repair rates. We did not run an agent, and a retrieval improvement need not survive contact with one. What the ceiling does license is the weaker, safer statement: a file absent from the context cannot be used, so the share of tasks whose answer fits the budget bounds any system operating under it. Section 6 specifies the experiment that would convert that bound into a performance claim, in advance of running it.

A note on order versus selection. An ordering plus a budget is a selection, and it is the selection that matters here. We make no claim about the position of a file *within* a prompt, which is a different question with its own literature; every result in this paper concerns which files are read at all.

The human reading of the same result. The budget for a developer is attention rather than tokens, and the reversal has a plain reading there too. Asked to fix the login page, a developer thinks of the authentication files because the words match, not because the imports connect — and the co-edited key, which records what a person actually chose to change, is where words win. The symbol key, which records what the compiler would care about, is where structure wins. We report the token framing because it makes the budget concrete and measurable; we do not think the finding is confined to machines.

Recommendations. (1) Report retrieval results separately for at least one structural and one behavioural notion of relevance, and report the worst case across them. (2) Treat a query-independent repository prior, not random order, as the floor for structural methods — on symbol-style keys it captures 92% of a personalised walk. (3) Include a lexical signal from the issue text in graph-based context tools. (4) Report cost in tokens and report the ceiling alongside the score, so that a reader can tell a hard task from a poor ordering.

6 A PRE-REGISTERED PROTOCOL FOR THE DOWNSTREAM TEST

This paper measures retrieval, not repair. The obvious next question — whether a context assembled by one of these orderings lets a model actually fix the issue — requires running models, which we have not done. Rather than gesture at it as future work, we specify it here in enough detail to be executed or contradicted, in the same spirit as the design freeze that preceded this study (§3). We commit to reporting the result whichever way it falls.

Corpus. The 1,162 scored pull requests of this study. Every one carries a test specification from the benchmark suite it came from, naming the tests that must change from failing to passing and those that must keep passing, so resolution can be decided by execution rather than by patch similarity. We note that only 15 of them overlap SWE-bench Verified, so this is not a re-run of an existing leaderboard.

Conditions. Six contexts per task, each truncated to the same token budget: (i) the seed file alone, a floor; (ii) seed plus the best structural ordering (ppr_und_pl); (iii) seed plus the best lexical ordering (path_issue); (iv) seed plus the best fusion (rrf_hops_path); (v) seed plus the true key, a ceiling; (vi) the repository truncated to the budget in a fixed canonical order, which is what a system with no retrieval step does. Budgets of 16,000 and 64,000 tokens, chosen from Table 8 to sit either side of the point where most answers begin to fit.

Models. Two, of clearly different capability, so that the result is not a property of one model. A weak model that fails everywhere and a strong model that succeeds everywhere would both be uninformative; the pilot below exists to detect that before the full run.

Generation. A single API call per task and condition, with no tools and no retrieval available to the model. This is essential rather than incidental: a model that can search the repository defeats the budget, and the quantity under study is what a fixed context supports.

Outcome and analysis. A task is resolved if the generated patch applies and the specified tests pass. The comparison is paired across conditions on the same tasks, tested with McNemar’s test, Holm-corrected across the condition pairs, with the pull request as the unit as elsewhere in this paper. We do not require the models to be reliable; we require the *difference between conditions* to be measurable, and the pairing makes the stochastic component common to all conditions.

Pilot and stopping rule. A 150-task pilot on the weaker model precedes the full run. If the oracle condition resolves under 10% or over 90% of pilot tasks, the budget is outside the informative range and is adjusted before the main run — and that adjustment is reported.

What would falsify the implication. If conditions (ii), (iii) and (iv) do not differ from (vi) at equal budget, then retrieval quality does not transfer to repair at these budgets, and the results of this paper should be read as being about retrieval alone. That outcome is publishable and we would report it.

7 THREATS TO VALIDITY

Construct. Neither evidence source is ground truth about what a developer needed to read; they are recoverable proxies with opposite biases, which is why we report both and never merge them. Coverage per line assumes whole files are read, which overstates the cost for agents that read selectively. The execution-based source is absent (§3.3) and would be the natural third leg.

Internal. The graph builder resolves imports statically and misses dynamic imports and plugin registration. The symbol key is produced by a static analyser and inherits its resolution failures. Both biases favour structural methods, so the co-edited reversal is if anything understated. Issue-path leakage is measured and shown not to drive the result.

External. Python only, file granularity only, and pull requests drawn from benchmark suites that select for reproducible test-verified fixes — a population plausibly better documented than average, which may flatter the issue-text methods. The median repository here has 131 graph files and very large repositories are under-represented, but the size split in §4.7 shows the reversal widening rather than narrowing with repository size, so the bias does not run in the direction that would explain it away.

Conclusion. Pull-request-level aggregation, Holm correction within comparison families and bootstrapping over repositories guard against the main inflation risks. Effect sizes accompany every test because several differences are significant and small.

8 CONCLUSION

We compared import-graph proximity and issue-text retrieval as reading orders for Python change tasks on 2,412 tasks from 1,162 pull requests across 112 repositories, under two evidence-based definitions of which files a change required. The comparison has no source-independent answer: the rankings the two sources induce over sixteen methods are statistically unrelated, with structural

orderings winning on files the change references and lexical orderings winning on files the developer changed alongside it. The graph’s two-hop neighbourhood, the standard justification for structural retrieval, turns out to be half the repository. Fusing a structural and a lexical ordering is the only strategy that stays near the top under both sources, and it costs almost nothing. We recommend that context-retrieval work for code change report per evidence source, use a repository prior rather than random order as its floor, and stop treating the dependency graph as the only signal in the room.

DATA AVAILABILITY

The replication package contains the import graphs at every base commit, the task definitions and evidence keys, the issue-token bags, the per-task orderings and coverage curves, the per-file token counts, and the analysis scripts that produce every number and figure in this paper. It will be archived at a permanent identifier on acceptance.

USE OF AI TOOLS

Claude (Anthropic) was used to write the analysis and method code, to run the experiments and to draft this manuscript. All numeric results are produced by scripts in the replication package and were copied mechanically from their output files; the author verified the design, the claims and the text. Bibliographic entries were drafted with AI assistance and then checked against arXiv, the ACL Anthology and the publisher listings on 2026-09-23; the evaluation section of every system in Table 1 was read directly rather than summarised from an abstract. That check corrected eight entries drafted from memory, including a journal name, an author order, three titles changed between preprint versions, and one volume recorded as authored that is in fact edited. The corrections are listed in the replication package.

REFERENCES

- [1] Aider. 2023. Building a Better Repository Map with Tree Sitter. <https://aider.chat/2023/10/22/repomap.html>. Blog post, 22 October 2023. Accessed 2026-09-23.
- [2] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2025. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. arXiv:2505.20411 [cs.SE]
- [3] Shawn A. Bohner and Robert S. Arnold (Eds.). 1996. *Software Change Impact Analysis*. IEEE Computer Society Press, Los Alamitos, CA.
- [4] Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, Jie Wang, Xiao Cheng, Guangtai Liang, Yuchi Ma, Pan Bian, Tao Xie, and Qianxiang Wang. 2024. CodeR: Issue Resolving with Multi-Agent and Task Graphs. arXiv:2406.01304 [cs.SE]
- [5] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. 2025. LocAgent: Graph-Guided LLM Agents for Code Localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 8697–8727. <https://doi.org/10.18653/v1/2025.acl-long.426> Introduces the Loc-Bench benchmark. arXiv:2503.09089.
- [6] Bogdan Dit, Meghan Revelle, Malcom Gethers, and Denys Poshyvanyk. 2013. Feature Location in Source Code: A Taxonomy and Survey. *Journal of Software: Evolution and Process* 25, 1 (2013), 53–95. <https://doi.org/10.1002/smr.567>
- [7] Kang He and Kaushik Roy. 2026. SWE-Adept: An LLM-Based Agentic Framework for Deep Codebase Analysis and Structured Issue Resolution. arXiv:2603.01327 [cs.SE]
- [8] Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. 2025. Issue Localization via LLM-Driven Iterative Code Graph Searching. arXiv:2503.22424 [cs.SE] Earlier version titled “CoSIL: Software Issue Localization via LLM-Driven Code Repository Graph Searching”. To appear, ASE 2025.
- [9] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world GitHub Issues?. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.06770. SWE-bench Lite is released with this benchmark.

- [10] Amy J. Ko, Brad A. Myers, Michael J. Coblenz, and Htet Htet Aung. 2006. An Exploratory Study of How Developers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks. *IEEE Transactions on Software Engineering* 32, 12 (Dec. 2006), 971–987. <https://doi.org/10.1109/TSE.2006.116>
- [11] Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. 2026. ContextBench: A Benchmark for Context Retrieval in Coding Agents. arXiv:2602.05892 [cs.SE]
- [12] Wei Liu, Chao Peng, Pengfei Gao, Aofan Liu, Wei Zhang, Haiyan Zhao, and Zhi Jin. 2025. GraphLocator: Graph-guided Causal Reasoning for Issue Localization. arXiv:2512.22469 [cs.SE]
- [13] Yingwei Ma, Qingping Yang, Rongyu Cao, Binhua Li, Fei Huang, and Yongbin Li. 2025. Alibaba LingmaAgent: Improving Automated Issue Resolution via Comprehensive Repository Exploration. In *Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25), Industry Track*. <https://doi.org/10.1145/3696630.3728549> arXiv:2406.01422 (v1 titled “How to Understand Whole Software Repository?”).
- [14] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. 2025. RepoGraph: Enhancing AI Software Engineering with Repository-Level Code Graph. In *International Conference on Learning Representations (ICLR)*. arXiv:2410.14684.
- [15] Bowen Qin and Yi Xie. 2026. Agent Retrieval Bench: Evaluating Repository Context Retrieval for Coding Agents. arXiv:2607.24882 [cs.SE]
- [16] Revanth Gangi Reddy, Tarun Suresh, JaeHyeok Doo, Ye Liu, Xuan Phi Nguyen, Yingbo Zhou, Semih Yavuz, Caiming Xiong, Heng Ji, and Shafiq Joty. 2025. SweRank: Software Issue Localization with Code Ranking. arXiv:2505.07849 [cs.SE]
- [17] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389. <https://doi.org/10.1561/15000000019>
- [18] Martin P. Robillard. 2008. Topology Analysis of Software Dependencies. *ACM Transactions on Software Engineering and Methodology* 17, 4 (Aug. 2008), 18:1–18:36. <https://doi.org/10.1145/13487689.13487691>
- [19] Lintang Sutawika, Aditya Bharat Soni, Bharath Sriraam R R, Apurva Gandhi, Taha Yassine, Sanidhya Vijayvargiya, Yuchen Li, Xuhui Zhou, Yilin Zhang, Leander Melroy Maben, and Graham Neubig. 2026. CodeScout: An Effective Recipe for Reinforcement Learning of Code Search Agents. arXiv:2603.17829 [cs.SE]
- [20] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE (July 2025), FSE037. <https://doi.org/10.1145/3715754> Agentless. arXiv:2407.01489.
- [21] Chengxing Xie, Bowen Li, Chang Gao, He Du, Wai Lam, Difan Zou, and Kai Chen. 2025. SWE-Fixer: Training Open-Source LLMs for Effective and Efficient GitHub Issue Resolution. In *Findings of the Association for Computational Linguistics: ACL 2025*. 1123–1139. <https://doi.org/10.18653/v1/2025.findings-acl.62> arXiv:2501.05040.
- [22] Boyang Yang, Jiadong Ren, Shunfu Jin, Yang Liu, Feng Liu, Bach Le, and Haoye Tian. 2025. Enhancing Repository-Level Software Repair via Repository-Aware Knowledge Graphs. arXiv:2503.21710 [cs.SE] KGCompass. An earlier version was titled “KGCompass: Knowledge Graph Enhanced Repository-Level Software Repair”.
- [23] Annie T. T. Ying, Gail C. Murphy, Raymond Ng, and Mark C. Chu-Carroll. 2004. Predicting Source Code Changes by Mining Change History. *IEEE Transactions on Software Engineering* 30, 9 (Sept. 2004), 574–586. <https://doi.org/10.1109/TSE.2004.52>
- [24] Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. 2025. SWE-bench Goes Live! arXiv:2505.23419 [cs.SE] NeurIPS 2025 Datasets and Benchmarks Track (per the authors’ repository).
- [25] Shaoqiu Zhang, Yuhang Wang, Jialiang Liang, Yuling Shi, Wenhao Zeng, Maoquan Wang, Shilin He, Ningyuan Xu, Siyu Ye, Kai Cai, and Xiaodong Gu. 2026. SWE-Explore: Benchmarking How Coding Agents Explore Repositories. arXiv:2606.07297 [cs.SE]
- [26] Thomas Zimmermann, Peter Weißerger, Stephan Diehl, and Andreas Zeller. 2004. Mining Version Histories to Guide Software Changes. In *Proceedings of the 26th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Edinburgh, United Kingdom, 563–572.